# logit_lens

October 6, 2026

## 1 Logit Lens — exploración inicial (2026-08-17)

**Objetivo de este notebook:** entender de forma práctica qué es el *logit lens* antes de aplicarlo como parte del bloque A (feature study) del capítulo de ViT — ver `CLAUDE.md`, punto **A.2**: *"proyectar el CLS token intermedio de cada bloque (post-LN) a través del clasificador final para rastrear en qué capa 'se decide' la clase"*.

No es todavía el análisis comparativo (baseline vs. penalizado) — es la prueba de concepto: cargar un modelo ya entrenado (uno de los checkpoints que ya existen en `experimentation/experiments/`) y ver, capa a capa, cómo evoluciona la predicción de clase.

### 1.1 ¿Qué es el logit lens?

La idea original es de **nostalgebraist (2020)**, *"interpreting GPT: the logit lens"* (post en LessWrong: lesswrong.com/posts/AcKRB8wDpdaN6v6ru): en un modelo tipo GPT, cada bloque transformer va sumando (vía conexiones residuales) al *residual stream*. La observación es que el embedding de salida (la matriz que convierte el estado oculto final en logits sobre el vocabulario) se puede aplicar **también a los estados intermedios**, no solo al final — como poner una "lupa" (lens) en cada capa para leer, en el espacio de logits, qué predeciría el modelo si se detuviera ahí. Esto revela que muchos modelos van "decidiéndose" gradualmente, capa a capa, en vez de decidir todo de golpe en la última capa.

Para un ViT de clasificación (no autoregresivo, sin vocabulario) el equivalente es: en vez de proyectar cada token contra el vocabulario, se proyecta el **token [CLS]** de cada bloque contra el **clasificador final** (la capa lineal que da los logits de clase). Es exactamente lo que hace este notebook. Esta adaptación está explícitamente estudiada en:

- Vilas, Schaumlöffel, Roig (NeurIPS 2023), *"Analyzing Vision Transformers for Image Classification in Class Embedding Space"* — arxiv.org/abs/2310.18969 · código. Proyectan representaciones intermedias de un ViT al espacio de embeddings de clase (misma idea que el logit lens, aplicada a visión) y muestran que la identificabilidad de la clase correcta crece progresivamente capa a capa, no solo en el token [CLS] sino también en los tokens de parche.
- Belrose et al. (2023), *"Eliciting Latent Predictions from Transformers with the Tuned Lens"* — arxiv.org/abs/2303.08112, la evolución del logit lens: en vez de reusar el clasificador final tal cual, entrena una pequeña transformación afín por capa para corregir el sesgo del logit lens "crudo". Buena referencia para cuando este notebook exploratorio se convierta en el análisis real del capítulo.
- Adjileye (Medium), *"Unlocking Visual Insights: Applying the 'Logit Lens' to Image Data with Vision Transformers"* (ya citado en `CLAUDE.md`) — versión aplicada y visual de lo mismo.

**Por qué es barato en esta arquitectura:** en `ViTForClassfication` (`modules/model.py`) el clasificador es una única capa lineal aplicada directamente al CLS del último bloque (`self.classifier(encoder_output[:, 0, :])`), sin LayerNorm final adicional. Eso significa que el logit lens es literalmente aplicar esa misma capa lineal al CLS de *cualquier* bloque intermedio — no hay que entrenar nada nuevo ni reimplementar el clasificador.

### 1.2 Qué hace este notebook

1. Carga un checkpoint ya entrenado (`vit-with-15-epochs-CIFAR10`, baseline sin penalización, ~60% accuracy — el más entrenado de los baseline disponibles, elegido porque queremos ver una progresión con señal real, no ruido de un modelo sin entrenar).
2. Implementa `logit_lens(modelo, x)`: para cada bloque (incluyendo el embedding como "capa 0"), toma el CLS y lo pasa por el clasificador final.
3. Verifica que la capa final del logit lens coincide exactamente con el forward normal del modelo (sanity check).
4. Mide accuracy por capa sobre un lote de imágenes de test — el gráfico central: ¿en qué bloque empieza a acertar la clase?
5. Mira ejemplos individuales: cómo evoluciona la probabilidad asignada a la clase correcta, capa a capa, y qué predice cada capa en concreto.

Cuando esto tenga sentido, el siguiente paso natural es repetir exactamente este notebook cargando el checkpoint **interpretable** (`vit-with-10-epochs-interpretable-CIFAR10`) y comparar en qué capa "se decide" cada variante — eso es ya el análisis real de A.2.

[1]:
```
import json
import sys
from pathlib import Path

# Igual que en notebooks/09082026_effective_rank/effective_rank.ipynb:
# esta notebook vive en notebooks/<algo>/, dos niveles debajo de la raíz del␣
 ↪proyecto.
project_root = Path.cwd().resolve().parents[1]
print(f"Project root: {project_root}")
sys.path.append(str(project_root))

import numpy as np
import matplotlib.pyplot as plt
import torch
import torch.nn.functional as F

from modules.model import ViTForClassfication
from modules.datasets import AwA2Dataset
from modules.figures import save_fig

torch.manual_seed(0)

EXPERIMENTS_DIR = project_root / "experimentation" / "experiments"
```

```
OUTPUT_DIR = Path.cwd()  # se guarda junto a este notebook, como documenta la␣
 ↪celda de abajo

MODELS = {
    "baseline": "vit-with-15-epochs-AwA2",
    "interpretable": "vit-with-15-epochs-interpretable-AwA2",
}
```

```
Project root: /Users/valefeve/Projects/interpretability of neural
networks/interpretable-transformers
```

**Todas las gráficas de este notebook se guardan solas** en esta misma carpeta, como `<nombre_de_la_grafica>_<nombre_del_modelo>.png` (sin que tengas que hacer nada extra) — para **ambos** modelos (`baseline` e `interpretable`) en cada corrida:

- `accuracy_por_capa_<modelo>.png` (sección 4)
- `probabilidad_por_ejemplo_<modelo>.png` (sección 5)
- `predicciones_por_capa_<modelo>.png` (sección 5)
- `matriz_confusion_<modelo>.png` (sección 6)

El nombre exacto con el que se guardó cada imagen se imprime justo debajo de la celda que la genera.

### 1.3 1. Cargar los dos modelos

Usamos los dos checkpoints entrenados sobre AwA2 (mismo config, misma seed de datos, ver `CLAUDE.md`): `vit-with-15-epochs-AwA2` (baseline, sin penalización) y `vit-with-15-epochs-interpretable-AwA2` (con penalización de similitud entre cabezas).

Cargamos con `map_location="cpu"` explícito para que corra igual sin importar si hay GPU/MPS disponible.

[2]:
```
def load_model(name):
    exp_dir = EXPERIMENTS_DIR / name
    with open(exp_dir / "config.json") as f:
        config = json.load(f)
    with open(exp_dir / "metrics.json") as f:
        metrics = json.load(f)
    model = ViTForClassfication(config)
    model.load_state_dict(torch.load(exp_dir / "model_final.pt",␣
 ↪map_location="cpu"))
    model.eval()
    return model, config, metrics


models = {key: load_model(name) for key, name in MODELS.items()}
config = models["baseline"][1]
assert config == models["interpretable"][1], "Los dos modelos deben tener la␣
 ↪misma arquitectura"
```

```
for key, (model, _, metrics) in models.items():
    print(f"{key:>13} ({MODELS[key]}): accuracy de entrenamiento =␣
 ↪{metrics['accuracy']:.4f}, "
          f"{len(model.encoder.blocks)} bloques")
```

```
    baseline (vit-with-15-epochs-AwA2): accuracy de entrenamiento = 0.3091, 4
bloques
interpretable (vit-with-15-epochs-interpretable-AwA2): accuracy de entrenamiento
= 0.3085, 4 bloques
```

### 1.4 2. Datos de test (AwA2)

`AwA2Dataset` usa el cache local ya construido en `data/awa2/`, así que no vuelve a descargar nada.

[3]:
```
dataset = AwA2Dataset(image_size=32, root=str(project_root / "data" / "awa2"))
_, testloader, classes = dataset.get_loaders(batch_size=256)
testset = dataset.testset
print(f"Test set: {len(testset)} imágenes, {len(classes)} clases")
```

```
Test set: 7468 imágenes, 50 clases
```

### 1.5 3. La función `logit_lens`

Corre el forward manualmente bloque por bloque (en vez de llamar `model(x)` de una), guardando el CLS token después de cada paso, y proyecta cada uno con el clasificador final. La "capa 0" es el CLS justo después de los embeddings (antes de cualquier bloque transformer) — sirve de referencia de qué tan bien predice el clasificador si no hubiera pasado por ningún bloque (debería estar cerca del azar, 10%).

[4]:
```
@torch.no_grad()
def logit_lens(model, x):
    """
    x: (B, C, H, W)
    Devuelve (layer_names, all_probs): all_probs es una lista de longitud
    num_hidden_layers + 1 con las probabilidades softmax (B, num_classes) de
    proyectar el CLS token a través del clasificador final, capturado después
    de los embeddings (capa 0) y después de cada bloque.
    """
    embedding_output = model.embedding(x)
    cls_states = [embedding_output[:, 0, :]]

    h = embedding_output
    for block in model.encoder.blocks:
        h, _ = block(h, output_attentions=False)
        cls_states.append(h[:, 0, :])

    layer_names = ["embeddings"] + [f"block {i+1}" for i in range(len(model.
 ↪encoder.blocks))]
    all_probs = [F.softmax(model.classifier(cls), dim=-1) for cls in cls_states]
```

```
    return layer_names, all_probs
```

[5]:
```
# Sanity check: la última capa del logit lens debe coincidir EXACTAMENTE
# con el forward normal del modelo, para cada uno de los dos.
sample_images, sample_labels = dataset.get_samples_from_indices(range(8),␣
 ↪device="cpu", set="test")

for key, (model, _, _) in models.items():
    layer_names, probs_per_layer = logit_lens(model, sample_images)

    with torch.no_grad():
        logits, _ = model(sample_images, output_attentions=False)
        direct_probs = F.softmax(logits, dim=-1)

    max_diff = (direct_probs - probs_per_layer[-1]).abs().max().item()
    assert max_diff < 1e-6, f"{key}: la última capa del logit lens debería ser␣
 ↪idéntica al forward normal"
    print(f"[{key}] diferencia máxima entre logit-lens (última capa) y forward␣
 ↪normal: {max_diff:.2e}")

print("OK - en ambos modelos, la última capa del logit lens reproduce␣
 ↪exactamente la predicción real.")
```

```
[baseline] diferencia máxima entre logit-lens (última capa) y forward normal:
0.00e+00
[interpretable] diferencia máxima entre logit-lens (última capa) y forward
normal: 0.00e+00
OK - en ambos modelos, la última capa del logit lens reproduce exactamente la
predicción real.
```

### 1.6 4. ¿En qué capa "se decide" la clase? — accuracy por capa

Esta es la pregunta central de A.2. Tomamos un lote de imágenes de test, corremos `logit_lens`, y en cada capa medimos qué fracción de las predicciones (top-1 del logit lens en esa capa) coincide con la etiqueta real. Si la curva sube rápido y se aplana antes del último bloque, quiere decir que el modelo "ya sabe" la clase antes de terminar — esa es la señal que se compara después entre baseline y penalizado.

[6]:
```
N_EVAL = 500  # tamaño del lote de evaluación (CPU-friendly)
eval_images, eval_labels = dataset.get_samples_from_indices(range(N_EVAL),␣
 ↪device="cpu", set="test")

for key, (model, _, metrics) in models.items():
    layer_names, probs_per_layer = logit_lens(model, eval_images)

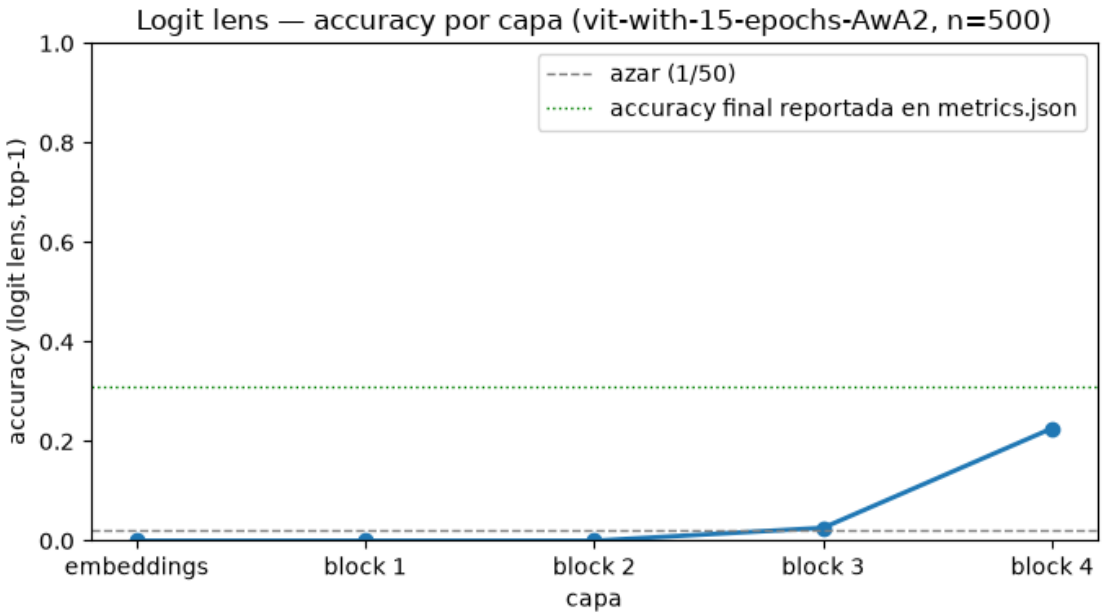
    layer_accuracies = []
    for probs in probs_per_layer:
        preds = probs.argmax(dim=-1)
```

```
        acc = (preds == eval_labels).float().mean().item()
        layer_accuracies.append(acc)

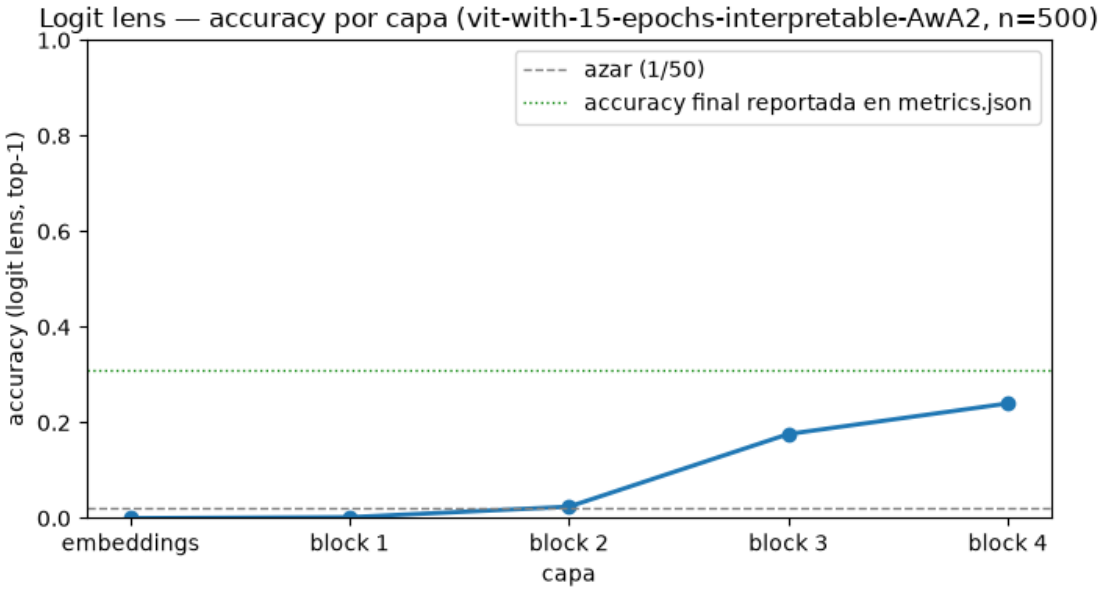
    for name, acc in zip(layer_names, layer_accuracies):
        print(f"[{key}] {name:>10}: accuracy = {acc:.3f}")

    fig, ax = plt.subplots(figsize=(7, 4))
    ax.plot(layer_names, layer_accuracies, marker="o", linewidth=2)
    ax.axhline(1 / len(classes), color="gray", linestyle="--", linewidth=1,␣
↪label=f"azar (1/{len(classes)})")
    ax.axhline(metrics["accuracy"], color="green", linestyle=":", linewidth=1,␣
↪label="accuracy final reportada en metrics.json")
    ax.set_ylabel("accuracy (logit lens, top-1)")
    ax.set_xlabel("capa")
    ax.set_title(f"Logit lens - accuracy por capa ({MODELS[key]}, n={N_EVAL})")
    ax.set_ylim(0, 1)
    ax.legend()
    plt.tight_layout()
    save_fig(fig, "accuracy_por_capa", OUTPUT_DIR, tag=MODELS[key],␣
↪project_root=project_root)
    plt.show()
    print()
```

```
[baseline] embeddings: accuracy = 0.000
[baseline]    block 1: accuracy = 0.000
[baseline]    block 2: accuracy = 0.000
[baseline]    block 3: accuracy = 0.026
[baseline]    block 4: accuracy = 0.226
guardado: notebooks/17082026_logit_lens/accuracy_por_capa_vit-with-15-epochs-
AwA2.png
```




```
[interpretable] embeddings: accuracy = 0.000
[interpretable]    block 1: accuracy = 0.002
[interpretable]    block 2: accuracy = 0.024
[interpretable]    block 3: accuracy = 0.176
[interpretable]    block 4: accuracy = 0.240
guardado: notebooks/17082026_logit_lens/accuracy_por_capa_vit-with-15-epochs-
interpretable-AwA2.png
```



### 1.7 5. Mirando ejemplos individuales

Para un puñado de imágenes: la probabilidad que el logit lens le asigna a la **clase verdadera** en cada capa (el gráfico clásico de logit lens), y qué clase predice top-1 en cada capa. Una imagen "fácil" debería subir rápido y quedarse arriba; una imagen mal clasificada al final puede mostrar dudas o cambios de opinión entre capas.

[7]:
```
N_EXAMPLES = 8
ex_images, ex_labels = dataset.get_samples_from_indices(range(N_EXAMPLES),␣
 ↪device="cpu", set="test")

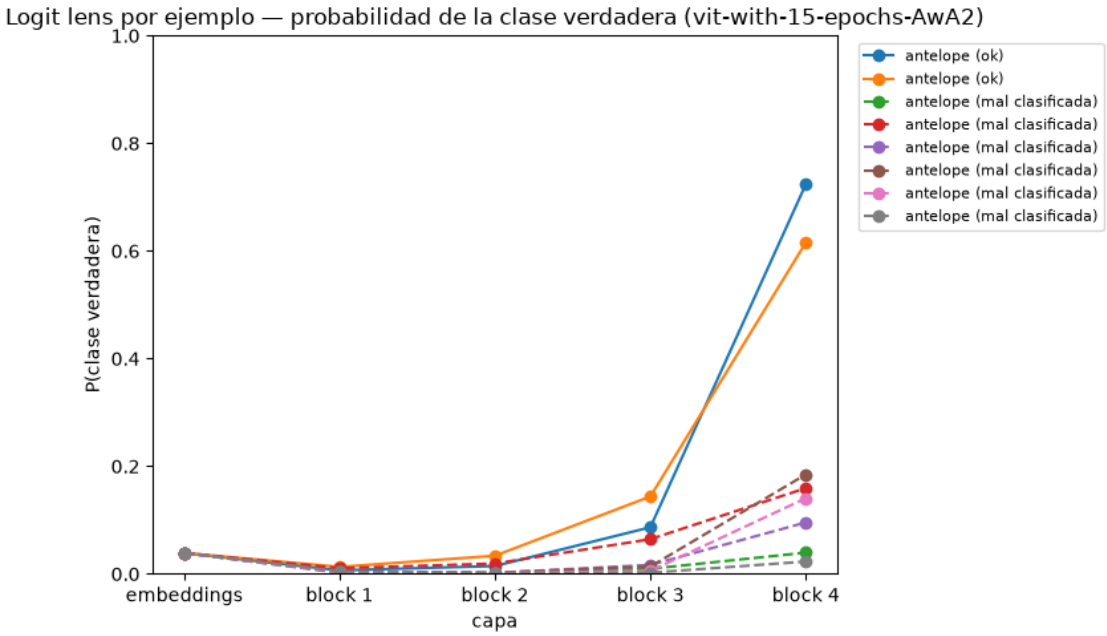
for key, (model, _, _) in models.items():
    layer_names, probs_per_layer = logit_lens(model, ex_images)

    # (n_layers, n_examples) -- probabilidad asignada a la clase verdadera en␣
 ↪cada capa
    true_class_probs = torch.stack([
        probs[torch.arange(N_EXAMPLES), ex_labels] for probs in probs_per_layer
    ])  # (n_layers, N_EXAMPLES)

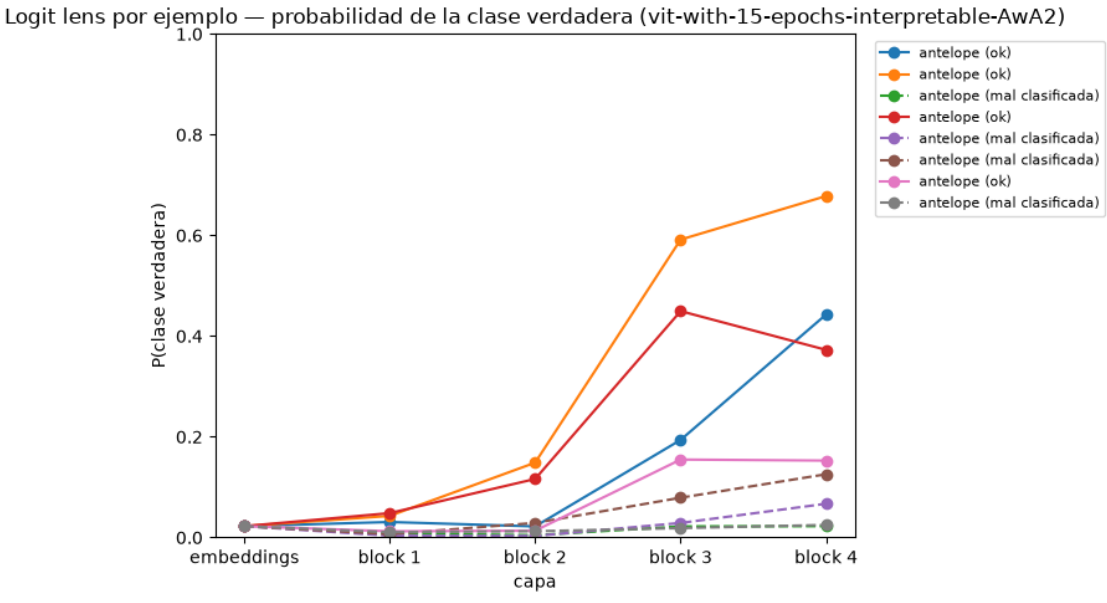
    fig, ax = plt.subplots(figsize=(8, 5))
    for i in range(N_EXAMPLES):
        final_pred_ok = (probs_per_layer[-1][i].argmax().item() == ex_labels[i].
 ↪item())
        style = "-" if final_pred_ok else "--"
        ax.plot(
            layer_names,
            true_class_probs[:, i].tolist(),
            marker="o",
            linestyle=style,
            label=f"{classes[ex_labels[i]]} ({'ok' if final_pred_ok else 'mal␣
 ↪clasificada'})",
        )
    ax.set_ylabel("P(clase verdadera)")
    ax.set_xlabel("capa")
    ax.set_title(f"Logit lens por ejemplo - probabilidad de la clase verdadera␣
 ↪({MODELS[key]})")
    ax.set_ylim(0, 1)
    ax.legend(fontsize=8, loc="upper left", bbox_to_anchor=(1.02, 1.0))
    plt.tight_layout()
    save_fig(fig, "probabilidad_por_ejemplo", OUTPUT_DIR, tag=MODELS[key],␣
 ↪project_root=project_root)
    plt.show()
```

```
    print()
```

```
guardado: notebooks/17082026_logit_lens/probabilidad_por_ejemplo_vit-
with-15-epochs-AwA2.png
```




```
guardado: notebooks/17082026_logit_lens/probabilidad_por_ejemplo_vit-
with-15-epochs-interpretable-AwA2.png
```



[8]:
```
# Grilla de imágenes con la secuencia de predicciones top-1 por capa, para verlo
# de forma muy concreta: "esto es lo que el modelo hubiera dicho si se detiene␣
 ↪aquí".
images_unnorm = ex_images * 0.5 + 0.5  # deshacer Normalize((0.5,...), (0.5,...
 ↪))

for key, (model, _, _) in models.items():
    layer_names, probs_per_layer = logit_lens(model, ex_images)

    ncols = 4
    nrows = (N_EXAMPLES + ncols - 1) // ncols
    fig, axes = plt.subplots(nrows, ncols, figsize=(ncols * 3, nrows * 3.2))
    axes = axes.flatten()

    for i in range(N_EXAMPLES):
        top1_per_layer = [classes[probs_per_layer[l][i].argmax().item()] for l␣
 ↪in range(len(layer_names))]
        sequence = " → ".join(top1_per_layer)
        true_label = classes[ex_labels[i]]
        axes[i].imshow(images_unnorm[i].permute(1, 2, 0).numpy())
        axes[i].set_title(f"real: {true_label}\n{sequence}", fontsize=7)
        axes[i].axis("off")

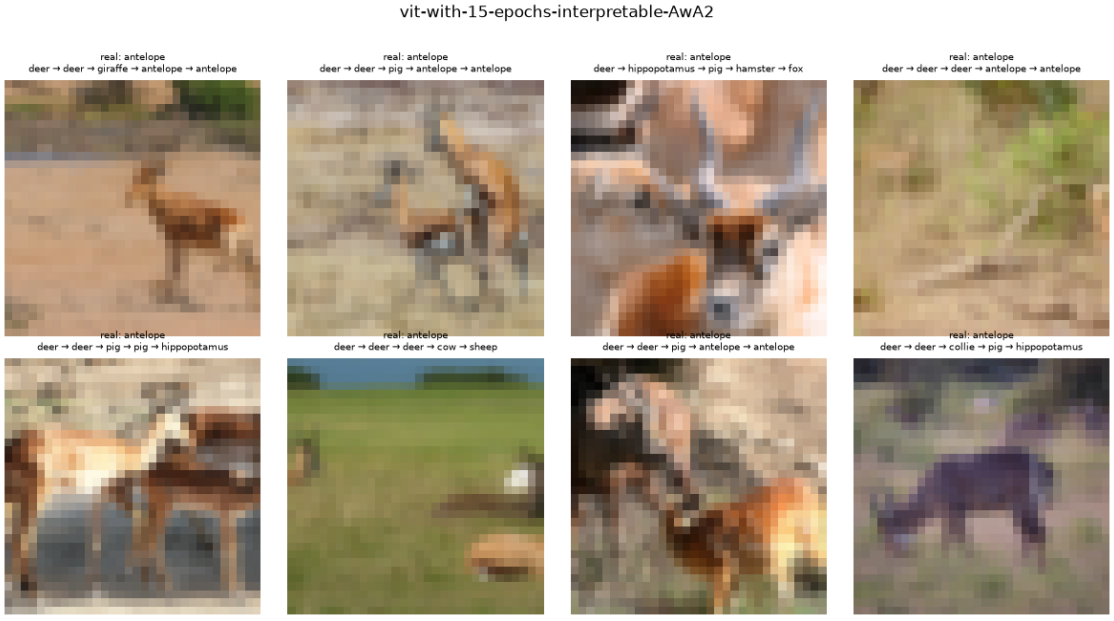
    for ax in axes[N_EXAMPLES:]:
        ax.axis("off")

    fig.suptitle(MODELS[key], y=1.02)
    plt.tight_layout()
    save_fig(fig, "predicciones_por_capa", OUTPUT_DIR, tag=MODELS[key],␣
 ↪project_root=project_root)
    plt.show()
    print()
```

guardado: notebooks/17082026_logit_lens/predicciones_por_capa_vit-with-15-epochs-AwA2.png




guardado: notebooks/17082026_logit_lens/predicciones_por_capa_vit-with-15-epochs-interpretable-AwA2.png



## 1.8 6. Matriz de confusión por capa (todo el set de evaluación, baseline y interpretable)

Para cada checkpoint (baseline y penalizado), y para cada capa (embeddings + cada bloque), la matriz de confusión de lo que predice el logit lens en esa capa — sobre **todo** el set de evaluación de CIFAR-10 (10 000 imágenes, no solo la muestra de `N_EVAL` que usan las secciones 4 y 5). Las matrices de un mismo checkpoint se combinan en una sola figura (una fila, una columna por capa) y se guardan como imagen **en esta misma carpeta**, con el nombre del checkpoint (sin extensión) — por ejemplo `vit-with-15-epochs-CIFAR10.png`.

No hace falta volver a cargar nada a mano: esta sección carga los dos checkpoints por su cuenta (usa una copia local del modelo, no toca la variable `model` de la sección 1) y genera los dos archivos en una sola corrida.

[9]:
```
CM_BATCH_SIZE = 500  # tamano de lote para no cargar las ~7500 imagenes de una


def confusion_matrix_np(true_labels, pred_labels, n_classes):
    cm = np.zeros((n_classes, n_classes), dtype=np.int64)
    np.add.at(cm, (true_labels, pred_labels), 1)
    return cm


def logit_lens_predictions_full(m, batch_size=CM_BATCH_SIZE):
    """Corre logit_lens sobre TODO el testset, en lotes, y devuelve
    (layer_names, true_labels_np, [preds_por_capa_np, ...])."""
    n_total = len(testset)
    all_true, preds_per_layer = [], None
    for start in range(0, n_total, batch_size):
        idx = range(start, min(start + batch_size, n_total))
        imgs, labels = dataset.get_samples_from_indices(idx, device="cpu",␣
 ↪set="test")
        names, probs = logit_lens(m, imgs)
        if preds_per_layer is None:
            preds_per_layer = [[] for _ in probs]
        for layer_idx, p in enumerate(probs):
            preds_per_layer[layer_idx].append(p.argmax(dim=-1))
        all_true.append(labels)
        print(f"  {min(start + batch_size, n_total)}/{n_total}", end="\r")
    print()
    true_labels_np = torch.cat(all_true).numpy()
    preds_per_layer_np = [torch.cat(p).numpy() for p in preds_per_layer]
    return names, true_labels_np, preds_per_layer_np

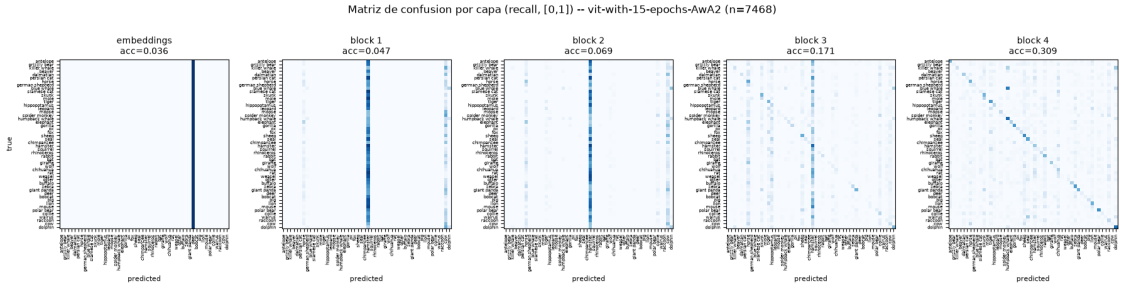

# Normalizada por fila (recall por clase, en [0, 1]) en vez de conteos crudos:␣
 ↪con
# conteos, imshow autoescala cada matriz a su propio máximo y el mismo color no
```

```
# significa lo mismo entre capas ni entre modelos. Con recall + escala fija␣
 ↪vmin=0,
# vmax=1 (mismo criterio que ya usa modules/training.py) sí es comparable a␣
 ↪simple vista.
for key, (model, _, _) in models.items():
    print(f"\n=== {MODELS[key]} ===")
    cm_layer_names, true_labels_np, preds_per_layer_np =␣
 ↪logit_lens_predictions_full(model)

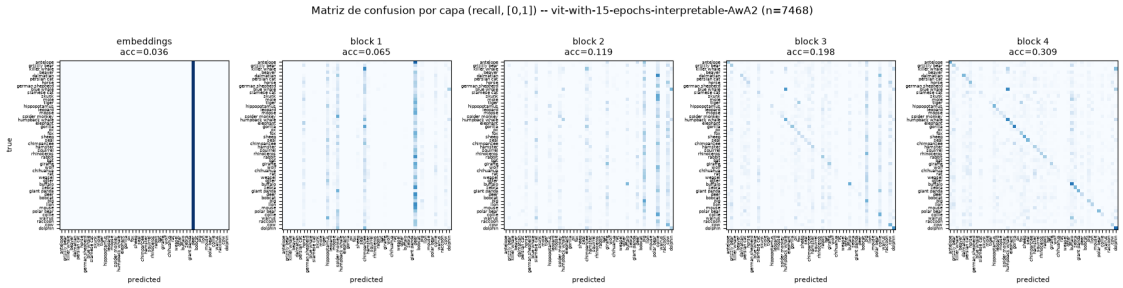
    n_layers = len(cm_layer_names)
    fig, axes = plt.subplots(1, n_layers, figsize=(3.6 * n_layers, 4.2))
    for ax, layer_name, preds in zip(axes, cm_layer_names, preds_per_layer_np):
        cm = confusion_matrix_np(true_labels_np, preds, len(classes)).
 ↪astype(float)
        cm_norm = cm / cm.sum(axis=1, keepdims=True).clip(min=1)
        acc = (true_labels_np == preds).mean()
        ax.imshow(cm_norm, cmap="Blues", vmin=0, vmax=1)
        ax.set_xticks(range(len(classes)))
        ax.set_xticklabels(classes, rotation=90, fontsize=5)
        ax.set_yticks(range(len(classes)))
        ax.set_yticklabels(classes, fontsize=5)
        ax.set_xlabel("predicted", fontsize=8)
        ax.set_title(f"{layer_name}\nacc={acc:.3f}", fontsize=10)
    axes[0].set_ylabel("true", fontsize=8)

    fig.suptitle(f"Matriz de confusion por capa (recall, [0,1]) --␣
 ↪{MODELS[key]} (n={len(testset)})", y=1.04, fontsize=12)
    plt.tight_layout()
    save_fig(fig, "matriz_confusion", OUTPUT_DIR, tag=MODELS[key],␣
 ↪project_root=project_root)
    plt.show()
```

```
=== vit-with-15-epochs-AwA2 ===
  7468/7468
guardado: notebooks/17082026_logit_lens/matriz_confusion_vit-with-15-epochs-
AwA2.png
```



```
=== vit-with-15-epochs-interpretable-AwA2 ===
  7468/7468
guardado: notebooks/17082026_logit_lens/matriz_confusion_vit-with-15-epochs-
interpretable-AwA2.png
```




**Qué mirar en las imágenes guardadas:** dentro de cada matriz, la diagonal es lo que el modelo acierta en esa capa — si ya está marcada en `block 2` o `block 3`, la clase se decide temprano. Fuera de la diagonal, los pares de clases que más se confunden entre sí — y si esa confusión aparece desde una capa temprana o solo se resuelve al final. Comparar `matriz_confusion_vit-with-15-epochs-AwA2.png` contra `matriz_confusion_vit-with-15-epochs-interpretable-AwA2.png` lado a lado es la comparación real que pide A.2 de `CLAUDE.md`.

Nota: con 50 clases (AwA2, vs. 10 de CIFAR-10) las anotaciones numéricas por celda se quitaron de la matriz de confusión — ilegibles a esta escala; solo queda el mapa de calor.

### 1.9 Para reflexionar / siguientes pasos

- ¿En qué capa deja de subir la accuracy agregada? ¿Es ya el bloque 2-3 de 4, o solo se decide en el último?
- ¿Las imágenes mal clasificadas al final ya se veían "dudosas" desde capas tempranas, o el modelo cambia de opinión tarde?
- Con un ViT de solo 4 bloques la ventana para ver progresión es corta — vale la pena repetir esto con más `N_EVAL` / más ejemplos individuales antes de sacar conclusiones.
- Las matrices de confusión de la sección 6 (baseline vs. interpretable, guardadas como imagen en esta carpeta) son la comparación real de A.2 — mira ahí si la penalización cambia en qué capa se decide la clase, y si cambia qué pares de clases se confunden entre sí.